

Guía de Implementación: Migración de SecureNet a Cloud

Checklist de Implementación

Fase 0: Preparación (Semana 1)

- ☐ Revisar documentacion (README, ARCHITECTURE, POWERUPS)
- ☐ Validar credenciales AWS (configurar `aws-cli`)
- ☐ Clonar repositorio `feature/terraform`
- ☐ Instalar Terraform localmente
- ☐ Generar SSH keypair para AWS
- ☐ Preparar variables en `terraform.tfvars`

Comandos:

```
# Instalar Terraform (Windows PowerShell)
choco install terraform # o descargar desde terraform.io

# Configurar AWS CLI
aws configure
# Ingresar: Access Key, Secret Key, region (eu-west-3)

# Generar SSH key (una sola vez)
ssh-keygen -t rsa -b 4096 -f ~/.ssh/id_jenkins

# Exportar clave pública
ssh-keygen -y -f ~/.ssh/id_jenkins > ~/.ssh/id_jenkins.pub
```

Fase 1: Despliegue en Laboratorios (Semana 1-2)

Paso 1: Inicialización de Terraform

```
cd terraform
terraform init
```

Resultado esperado:

```
✓ Terraform has been successfully initialized!
```

Paso 2: Planificación

```
terraform plan -out=tfplan
```

Revisar salida:

- Debe mostrar `aws_instance.jenkins_aws`
- Debe mostrar `aws_security_group.jenkins`
- Debe mostrar `aws_key_pair.ssh_key`

Paso 3: Aplicación

```
terraform apply tfplan
```

Verificar:

- ☐ Instancia Jenkins creada en EC2
- ☐ Security group configurado
- ☐ IP pública asignada
- ☐ Estado Terraform guardado como `terraform.tfstate`

Paso 4: Validación

```
# Obtener IP pública
terraform output ip_public_jenkins_aws

# SSH a la instancia
ssh -i ~/.ssh/id_jenkins ec2-user@<public-ip>

# Verificar logs de bootstrap
cat /var/log/user-data.log

# Verificar Jenkins running
sudo docker ps

# Verificar healthcheck
cat /opt/jenkins/.user_data_ran
```

Paso 5: Acceso a Jenkins

```
Abrir navegador: http://<public-ip>:8080

Obtener password inicial:
cat /opt/jenkins/docker-compose.yaml
# O dentro del contenedor:
sudo docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

Fase 2: Configuración de Jenkins (Semana 2-3)

Paso 1: Setup Inicial

1. Copiar contraseña de `initialAdminPassword`
2. Seleccionar "Install suggested plugins"
3. Crear usuario admin
4. Salvar configuración

Paso 2: Configurar Credenciales Git

1. Jenkins → Manage Jenkins → Manage Credentials
2. Add Credentials → SSH Key
3. Ingresar: Username (git), Private Key (id_jenkins contenido)

Paso 3: Crear First Pipeline

```
pipeline {
  agent any

  stages {
    stage('Clone') {
      steps {
        git credentialsId: 'git-ssh',
          url: 'git@github.com:Sniifeerns/securenet.git',
          branch: 'feature/terraform'
      }
    }

    stage('Terraform Plan') {
      steps {
        dir('terraform') {
          sh 'terraform plan'
        }
      }
    }
  }
}
```

Fase 3: Preparación Segunda VM (Semana 3)

Paso 1: Crear Script `docker_app.sh`

```
# terraform/scripts/docker_app.sh
```

```
#!/bin/bash
set -euxo pipefail

# Log user-data output
exec > >(tee /var/log/user-data.log | logger -t user-data -s) 2>&1

# Update system
sudo dnf update -y
sudo dnf install -y docker git curl

# Enable Docker
sudo systemctl enable --now docker
sudo usermod -aG docker ec2-user

# Install Docker Compose plugin
if ! sudo docker compose version >/dev/null 2>&1; then
    sudo mkdir -p /usr/local/lib/docker/cli-plugins
    sudo curl -SL
    "https://github.com/docker/compose/releases/latest/download/docker-compose-linux-
    x86_64" \
        -o /usr/local/lib/docker/cli-plugins/docker-compose
    sudo chmod +x /usr/local/lib/docker/cli-plugins/docker-compose
fi

# Create app directory
sudo mkdir -p /opt/app

# Generate docker-compose for app services
sudo tee /opt/app/docker-compose.yaml > /dev/null << 'EOF'
version: '3.8'

services:
    # Placeholder para aplicaciones
    # Serán desplegadas via Jenkins CI/CD

volumes:
    app_data:
EOF

echo "$(date -Is) user_data completed" | sudo tee /opt/app/.user_data_ran >
/dev/null
```

Paso 2: Crear recurso EC2 para Apps en Terraform

```
# En aws_compute.tf

resource "aws_instance" "app_aws" {
    ami = data.aws_ami.amazon_linux.id
    instance_type = "t3.medium"
    tags = {
        Name = "app-aws"
    }
}
```

```

    }
    user_data = file("${path.module}/scripts/docker_app.sh")
    user_data_replace_on_change = true
    vpc_security_group_ids = [aws_security_group.app.id]
    key_name = aws_key_pair.ssh_key.key_name
}

resource "aws_security_group" "app" {
  name          = "app"
  description   = "Security group for App instance"

  ingress {
    from_port    = 22
    to_port      = 22
    protocol     = "tcp"
    cidr_blocks  = ["0.0.0.0/0"] # LABS only
  }

  # Puertos para aplicaciones
  ingress {
    from_port    = 3000
    to_port      = 3000
    protocol     = "tcp"
    cidr_blocks  = ["0.0.0.0/0"]
  }

  ingress {
    from_port    = 5432
    to_port      = 5432
    protocol     = "tcp"
    cidr_blocks  = ["172.31.0.0/16"] # VPC CIDR (DB privada)
  }

  egress {
    from_port    = 0
    to_port      = 0
    protocol     = "-1"
    cidr_blocks  = ["0.0.0.0/0"]
  }
}

```

Paso 3: Desplegar

```

terraform plan
terraform apply

# Validar
terraform output ip_public_app_aws
ssh -i ~/.ssh/id_jenkins ec2-user@<app-ip>
cat /var/log/user-data.log

```

Fase 4: Implementar Power Apps (Mes 1)

Paso 1: Crear app en Power Apps

1. Crear Canvas App web en Power Apps.
2. Anadir una vista de estado de instancias (Jenkins y App).
3. Anadir una vista de estado de contenedores.

Paso 2: Crear flujos en Power Automate

1. Flujo para consultar estado de instancias.
2. Flujo para consultar estado de contenedores.
3. Flujo para acciones basicas: start, stop y restart.

Paso 3: Integrar acciones basicas

1. Conectar Power Automate con endpoints o scripts controlados en AWS.
2. Mostrar resultado en Power Apps (exito/fallo).
3. Registrar acciones basicas para auditoria.

Paso 4: Validacion

1. Verificar que la app muestra estado real.
 2. Reiniciar Jenkins desde la app y confirmar recuperacion.
 3. Verificar que acciones no autorizadas se bloquean.
-

Fase 5: Testing de Disaster Recovery (Semana 4)

Paso 1: Simular fallo

```
# SSH a Jenkins
ssh ec2-user@<jenkins-ip>

# Simular crash
sudo docker stop jenkins

# Verificar estado
sudo docker ps
```

Paso 2: Recuperación

```
# Reiniciar manualmente
sudo docker start jenkins
```

```
# 0 via Power Apps  
# (accion Restart desde la interfaz web)
```

Paso 3: Validar

```
curl http://<jenkins-ip>:8080/api/json  
  
# Debe retornar JSON (200 OK)
```

Fase 6: Preparación Migración a Producción (Mes 2)

Paso 1: Auditoría de Seguridad

- ☐ Review Security Groups (restringir 0.0.0.0/0)
- ☐ Habilitar VPC Flow Logs
- ☐ Configurar CloudTrail
- ☐ Implementar backup de Jenkins Home volume

Paso 2: Configurar Backend Remoto

```
# backend.tf  
terraform {  
  backend "s3" {  
    bucket      = "securenet-terraform-state"  
    key         = "jenkins/terraform.tfstate"  
    region      = "eu-west-3"  
    encrypt     = true  
    dynamodb_table = "terraform-locks"  
  }  
}
```

Paso 3: Crear cuenta AWS Producción

- ☐ Nueva AWS Account
- ☐ Configurar AWS Organizations
- ☐ Setup IAM roles
- ☐ Configurar billing alerts

Paso 4: Validar Terraform en Producción

```
# Con credenciales de producción  
export AWS_PROFILE=production
```

```
terraform init
terraform plan -var-file=production.tfvars
```

Troubleshooting Commons

Terraform init falla

```
# Error: No authentication via AWS CLI
aws configure
# Ingresar credenciales

# Error: Provider download falló
rm -rf .terraform
terraform init
```

SSH connection refused

```
# Verificar security group
aws ec2 describe-security-groups --group-names jenkins

# Verificar key permissions
chmod 600 ~/.ssh/id_jenkins

# Test conexión
ssh -i ~/.ssh/id_jenkins -v ec2-user@<ip>
```

Jenkins no levanta

```
ssh ec2-user@<ip>

# Ver logs
tail -f /var/log/user-data.log

# Ver Docker logs
sudo docker logs jenkins

# Reiniciar
sudo docker restart jenkins

# Ver healthcheck
cat /opt/jenkins/.user_data_ran
```

Power Apps permission denied

```
# Problema: usuario no está en grupo docker
sudo usermod -aG docker ec2-user

# Requiere logout/login
exit
ssh ec2-user@<ip>

# Test
docker ps
```

Documentos Relacionados

- [README.md](#) - Visión general de la migración
- [ARCHITECTURE.md](#) - Diseño técnico detallado
- [POWERUPS.md](#) - Roadmap de Power Apps para operaciones basicas

Contacto y Preguntas

- Repository: <https://github.com/Sniifeerns/securenet>
- Branch: [feature/terraform](#)
- Issues: GitHub Issues

Versión del Documento

- **Version:** 1.0
- **Fecha:** Marzo 2026
- **Autor:** SecureNet DevOps Team
- **Status:** Ready for Labs Phase 1